

Luma Visual Automation Fix Plan

A seriously practical path from research to code to shipping trust.

Page 1 - the real approach

Nox, the fix is not "make the model smarter" first.

The fix is to build a small automation operating system inside Luma. Right now Luma has powerful parts: ``LumaImageProcessingEngine`` finds things, ``CursorGuide`` points, ``LumaFlowActions`` clicks/types/scrolls, ``LumaFlowEngine`` replans, and ``WalkthroughEngine`` validates learning steps. But the power is spread across too many places. That is why visual automation feels hard to tune. You are tuning five systems when you think you are tuning one.

So the move is this: create one central ``VisualAutomationKernel`` and make every visual action pass through it.

Not because architecture is cute. Because trust is the product.

The kernel should own the full loop: inspect, resolve target, choose action policy, perform one atomic action, verify, recover. If Luma points, clicks, types, scrolls, launches, sends, opens, or validates, it should leave a transaction trail. We should be able to replay the question: what did Luma see, what did it think the target was, what coordinate space was used, what action path was chosen, what changed after, and why did it claim success?

Research phase comes first. For 2-3 days, do not code huge features. Collect evidence. Run 20 real tasks manually with Luma logging turned on: open Finder, open Downloads, click Xcode build, search Beeper contact, type into a field, open System Settings permission pane, guide through a browser form. For each failure, label it. Was it coordinate conversion? stale AX element? wrong frontmost app? missing AX label? model hallucinated target? screenshot downscaling? user focus changed? verification too weak?

This is the actual fine-tuning loop. Not gradient descent. Not magic. Just failure taxonomy until the pattern becomes obvious.

The language of the code should become stricter. Luma should stop passing naked ``CGPoint`` around like it is harmless. A point is never just a point. It came from a screenshot, an AX frame, a display, a scale factor, a window, a coordinate system, and a time. So create typed coordinates: ``ScreenshotPixelFormat``, ``DisplayPoint``, ``QuartzClickPoint``, ``AppKitOverlayPoint``. Make the compiler yell when you try to use an overlay point for a real click.

First implementation target:

```
struct UISnapshot: Identifiable {
    let id: UUID
    let createdAt: Date
    let bundleID: String
    let pid: pid_t
    let windowTitle: String?
    let displayID: CGDirectDisplayID
    let screenshot: ScreenCaptureMetadata?
    let axElements: [AXElementSnapshot]
    let focusedElement: AXElementSnapshot?
}

struct AutomationTransaction: Identifiable {
    let id: UUID
    let userGoal: String
    let snapshotID: UUID
    let resolvedTarget: ResolvedTarget?
    let action: AutomationAction
    let result: ActionResult?
}
```

That is the spine.

Page 2 - the dev process

Development should happen in four passes.

Pass 1: coordinate truth. Add ``CoordinateSpace.swift``. Move every conversion from ``CursorGuide``, ``LumaImageProcessingEngine``, ``CompanionScreenCaptureUtility``, and ``LumaFlowActions`` into it. Add tests for single display, Retina display, secondary display above/below/left/right, screenshot downscaling, and AX frame conversion. Until this exists, every visual bug can masquerade as an AI bug. That wastes your life.

Pass 2: snapshot before action. Add a ``UISnapshotBuilder`` that captures the current app/window state: PID, bundle ID, window title, display metadata, screenshot metadata, ranked AX elements, focused element, timestamp. Then change ``LumaFlowActions`` so it does not search the world directly. It asks the kernel for a snapshot, resolves a target from that snapshot, and acts based on that resolved target. Snapshot expires quickly. If the UI changes, new snapshot.

Pass 3: action safety. Add an ``AutomationActionRisk`` enum. Safe actions can happen automatically: read UI, point overlay, AXPRESS on verified element, AX value set on verified field. Medium actions can happen when bound to a snapshot: focus verified field then type, app-scoped shortcut, scroll verified region. Risky actions need explicit user intent or confirmation: global CGEvent click, ``clickAt``, shell command, file delete, send message, account change. This is where Luma starts feeling adult.

Pass 4: verification. Stop returning strings like "Clicked button" as the real result. Return ``ActionResult``. Verification should be structured: ``elementExists``, ``valueEquals``, ``windowTitleContains``, ``fileExists``, ``messageBubbleExists``, ``screenRegionChanged``, ``appActivated``. A screenshot hash is useful as a hint, not proof. Proof is tied to the user goal.

Suggested code shape:

```
enum AutomationActionRisk: String, Codable {
    case readOnly
    case safeUI
    case mediumUI
    case riskyUI
    case destructive
}

struct ActionResult: Codable {
    let actionID: UUID
    let succeeded: Bool
    let risk: AutomationActionRisk
    let evidenceSummary: String
    let verification: VerificationResult?
    let retryRecommendation: RetryRecommendation?
    let userVisibleMessage: String
}
```

The process should be test-driven but not slow. Build a tiny local eval harness. No huge lab. Just JSON fixtures. Each task has: starting app, goal, allowed actions, forbidden actions, success verifier, timeout. Run 25 tasks every time you touch automation. Track pass rate. Track wrong-target rate. Track retries. Track average latency. Luma needs a scoreboard.

The first benchmark should be brutally practical: Finder create folder, Finder reveal file, browser open URL, Xcode find build button, System Settings open Accessibility pane, Beeper search contact, Beeper type message without sending, Notes create note, Terminal run pwd in a managed session, and walkthrough "explain this UI" without clicking.

If a task fails, you do not patch randomly. You classify the failure. Then fix the layer. Coordinate failure? Fix ``CoordinateSpace``. AX label failure? Fix resolver scoring. Risk policy failure? Fix action policy. Verification failure? Fix verifier. Model bad action? Improve schema/prompt only after system layers are proven.

Page 3 - the language, UX, and shipping plan

The language inside Luma matters. The app should speak like a calm senior friend, not like a terminal log. But internally, the code should be strict and boring. User copy can be warm. Runtime language must be exact.

For the user, say: "I found the Send button in Enoch Computer Science on WhatsApp. Ready to send?" For the runtime, store: target bundle, target window, AX role, AX title, snapshot ID, confidence, action risk, verifier. Warm outside. Ruthless inside.

The UI should expose confidence without making the user read logs. High confidence: cursor points directly. Medium confidence: highlight a region and say "I think this is it." Low confidence: ask. Blocked: say what is missing. Done: say what proof was found. This is how Luma becomes trustworthy.

The model prompts should shrink. Do not make the model your safety system. The model proposes one action. The kernel approves or rejects it. The prompt should say: output JSON only, one action only, no explanation, include expected observation. The code decides whether the action is allowed.

The shipping plan:

Week 1: coordinate truth and typed points. No more raw ``CGPoint`` across subsystem boundaries. Add tests. Migrate overlay pointing and real clicking separately.

Week 2: ``UISnapshot`` and resolver. One snapshot object. One ranked element list. One target resolver. Merge duplicate AX search logic from ``CursorGuide`` and ``LumaImageProcessingEngine``.

Week 3: action policy and ``ActionResult``. Make ``LumaFlowActionExecutor`` return structured results. Add risk gates. Add confirmation for risky paths. Add verifier types.

Week 4: eval harness and 25-task scorecard. Run it. Break it. Fix top failure class. Repeat until Luma has receipts.

Do not try to beat HeyClicky everywhere in one sprint. Beat one lane: screen-aware teaching plus safe visual actions. That is where Luma has soul. HeyClicky can execute tasks. Luma can teach the task, explain the screen, guide the hand, and remember the learner.

That is the win.

Build the kernel. Make the compiler protect you. Make every action prove itself. Then let the companion bubble be beautiful on top of something strong.

Maybe Luma flops on the first benchmark. Cool. That is data. Run it again. Fix one layer. Run it again. This is how the messy solo-builder thing becomes real software.

We go again.